

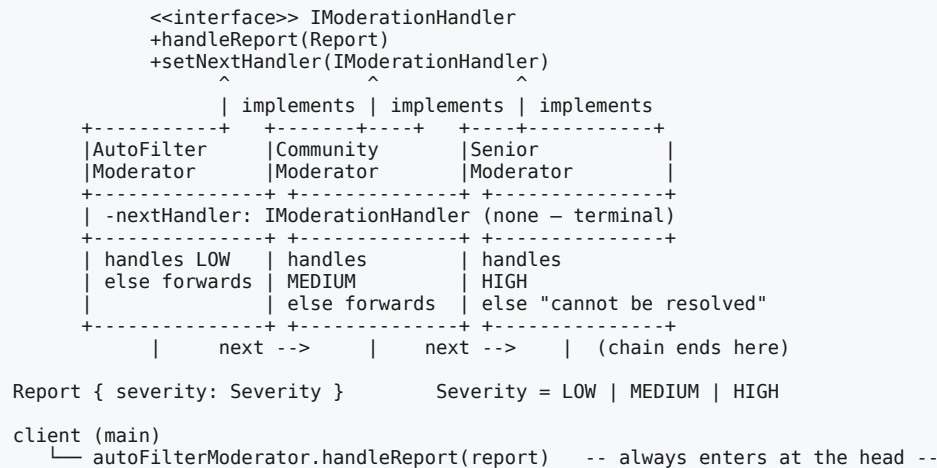
Chain of Responsibility Design Pattern

Intent

Decouple the **sender** of a request from its **receiver** by giving more than one object a chance to handle it. The request travels down a **chain** of handlers; each handler either deals with it or passes it to the next one. The sender doesn't know — and doesn't care — which handler ends up doing the work.

Here the chain models a **content-moderation pipeline**: a flagged post's `Report` carries a `Severity`, and is passed to the automated filter, which either resolves it or escalates to the community moderator, who either resolves it or escalates to the senior moderator — the fixed end of the chain.

UML class diagram



The players

report/Severity report/Report	LOW, MEDIUM, HIGH – the routing key the thing being passed along (carries a Severity)
handler/IModerationHandler handler/impl/AutoFilterModerator handler/impl/CommunityModerator handler/impl/SeniorModerator	the handler contract: handleReport + setNextHandler handles LOW, else escalates handles MEDIUM, else escalates handles HIGH, else "cannot be resolved" (chain end)
ChainOfResponsibilityDesignPattern	wires the chain and fires three reports (LOW, MEDIUM, HIGH)

Code walkthrough

Severity — the report classification

```
public enum Severity {
    LOW, MEDIUM, HIGH
}
```

- `enum Severity { LOW, MEDIUM, HIGH }` — three constants, one per moderation tier. *Why an enum*: it's a closed, type-safe set of values — a handler's `if (report.getSeverity() == Severity.X)` check can never be handed a typo or an out-of-range value the way a `String` or `int` could. This enum is effectively the routing key that decides *who* reviews a report.

Report — the message travelling the chain

```

public class Report {
    private Severity severity;

    public Report(Severity severity) {
        this.severity = severity;
    }

    public Severity getSeverity() {
        return severity;
    }
}

```

- `private Severity severity;` — the one piece of data the chain routes on. *Why private:* encapsulation — nothing outside `Report` can mutate the severity once set, so a report's classification can't change mid-flight through the chain.
- `public Report(Severity severity) { this.severity = severity; }` — the only way to set `severity`, and only at construction time. *Why:* this makes a `Report` effectively immutable — every handler along the chain is guaranteed to see the same severity the sender created it with.
- `public Severity getSeverity() { return severity; }` — the sole accessor. *Why:* it's the only thing a handler needs to know about a report to make its handle-or-forward decision; a real system would add more fields (post id, flagged text, reporter id, ...), but `severity` alone is enough to demonstrate routing.

IModerationHandler — the handler contract

```

public interface IModerationHandler {
    void handleReport(Report report);

    void setNextHandler(IModerationHandler nextHandler);
}

```

This is the **essence of the pattern**, deliberately kept to just two methods:

- `void handleReport(Report report);` — "try to resolve this; if you can't, pass it on." *Why it takes the whole Report and returns nothing:* the handler either fully consumes the report (prints a "resolved" message) or fully delegates it — there's no partial result to return, so `void` is the honest signature.
- `void setNextHandler(IModerationHandler nextHandler);` — lets you **link** one handler to the next, forming the chain. *Why the parameter is typed as the interface* (not a concrete class): any handler can point to any other handler — the links are interchangeable, and this handler compiles without ever importing a concrete sibling class.

AutoFilterModerator

```

public class AutoFilterModerator implements IModerationHandler {
    private IModerationHandler nextHandler;

    @Override
    public void handleReport(Report report) {
        if (report.getSeverity() == Severity.LOW) {
            System.out.println("Auto-filter moderator resolved the report.");
        } else if (nextHandler != null) {
            System.out.println("-----Escalating report to next moderator: " +
nextHandler.getClass().getName());
            nextHandler.handleReport(report);
        }
    }

    @Override
    public void setNextHandler(IModerationHandler nextHandler) {
        this.nextHandler = nextHandler;
    }
}

```

- `private IModerationHandler nextHandler;` — the link to the *next* handler in the chain. *Why the interface type:* this handler has no idea what concrete class comes after it — that's the decoupling the pattern is built on.
- `if (report.getSeverity() == Severity.LOW)` — "can I resolve it?" test. *Why this is first:* a handler always gets the chance to claim the report before considering escalation — that's the "responsibility" each link in the chain owns.
- `System.out.println("Auto-filter moderator resolved the report.")` — the report stops here; no forwarding happens once a handler claims it. *Why:* this is the *pure* form of Chain of Responsibility, where exactly one handler consumes a given report.
- `else if (nextHandler != null)` — the escalation branch, guarded against being the end of the chain. *Why the null check:* without it, a handler with no successor set would call a method on `null` and throw `NullPointerException` instead of failing gracefully.

- `System.out.println("-----Escalating report to next moderator: " + nextHandler.getClass().getName());` — a trace line naming the concrete class being escalated to. *Why*: it makes the otherwise-invisible hand-off between handlers visible in the console output, which is the whole point of a demo.
- `nextHandler.handleReport(report);` — delegates the *same* Report object downstream, unchanged. *Why the same object*: the chain doesn't transform the report, only decides who acts on it.
- `setNextHandler` — stores whatever successor is passed in. *Why it's this simple*: linking is a pure wiring operation with no validation needed — any `IModerationHandler` is a legal successor.

`CommunityModerator` is identical in shape, but its "can I resolve it?" test is `Severity.MEDIUM`, its success message is "Community moderator resolved the report.", and it escalates to whatever `nextHandler` it was given (in this demo, `SeniorModerator`).

SeniorModerator — the end of the chain

```
public class SeniorModerator implements IModerationHandler {
    @Override
    public void handleReport(Report report) {
        if (report.getSeverity() == Severity.HIGH) {
            System.out.println("Senior moderator resolved the report.");
        } else {
            System.out.println("Report could not be resolved by any moderator.");
        }
    }

    @Override
    public void setNextHandler(IModerationHandler nextHandler) {
    }
}
```

Two things make this the **terminal** handler:

- **No nextHandler field, and setNextHandler(...) has an empty body.** *Why*: there is nothing after the senior moderator in this demo, so the setter simply discards whatever is passed to it — you cannot link anything after this handler, by construction.
- **The else branch prints "Report could not be resolved by any moderator." instead of forwarding.** *Why*: this is the chain's fallback. If a report reaches the end and still nobody has claimed it, the chain reports failure explicitly rather than silently dropping it. (In this module every severity has a matching handler, so this branch is reachable only if `Severity` ever grew a fourth constant.)

ChainOfResponsibilityDesignPattern — building and firing the chain

```
public class ChainOfResponsibilityDesignPattern {
    public static void main(String[] args) {
        System.out.println("Chain Of Responsibility Design Pattern");

        AutoFilterModerator autoFilterModerator = new AutoFilterModerator();
        CommunityModerator communityModerator = new CommunityModerator();
        SeniorModerator seniorModerator = new SeniorModerator();

        autoFilterModerator.setNextHandler(communityModerator);
        communityModerator.setNextHandler(seniorModerator);

        System.out.println("-- LOW severity report --");
        autoFilterModerator.handleReport(new Report(Severity.LOW));

        System.out.println("-- MEDIUM severity report --");
        autoFilterModerator.handleReport(new Report(Severity.MEDIUM));

        System.out.println("-- HIGH severity report --");
        autoFilterModerator.handleReport(new Report(Severity.HIGH));
    }
}
```

- `System.out.println("Chain Of Responsibility Design Pattern");` — a banner line identifying the demo, printed once before any chain activity.
- **Create the three handlers** — `new AutoFilterModerator()`, `new CommunityModerator()`, `new SeniorModerator()`. *Why three concrete instances here and nowhere else*: `main` is the **only** place in the module that is allowed to know all three concrete classes exist — every handler class only ever refers to `IModerationHandler`.
- `autoFilterModerator.setNextHandler(communityModerator);` `communityModerator.setNextHandler(seniorModerator);` — links them into `autoFilter` → `community` → `senior`. *Why wired here*: building the chain's shape is a client/composition-root concern, not something a handler should decide about itself.
- **Three report/print pairs** (`-- LOW severity report --`, `-- MEDIUM severity report --`, `-- HIGH severity report --`), each followed by `autoFilterModerator.handleReport(new Report(severity))`. *Why all three, and why always through autoFilterModerator*: firing every severity through the same, single entry point is what proves the

pattern — the sender's call site is identical in all three cases, yet a different handler ends up doing the work each time. The sender never names the community or senior moderator directly.

Why these design decisions

Why Chain of Responsibility at all?

Without it, the sender would need a big conditional:

```
if (severity == LOW)      autoFilter.handle();
else if (severity == MEDIUM) community.handle();
else if (severity == HIGH) senior.handle();
```

That couples the sender to **every** handler and every routing rule. With the chain, the sender calls **one** method on **one** handler and the routing logic lives distributed across the handlers themselves. You can reorder, insert, or remove handlers without ever touching the sender.

Why is the interface only two methods (`handleReport` + `setNextHandler`)?

This is the smallest contract that still expresses the pattern: one method to *attempt/handle*, one to *link the successor*. Keeping it minimal means every handler is trivial to implement and any handler can be chained to any other — no base class, no builder, no registry, just the two operations that define the pattern.

Why does each handler hold a reference to the *next* one, typed as the interface?

So the chain is a runtime-configurable linked list. Typing the link as `IModerationHandler` (not a concrete class) is what decouples the handlers from one another — the auto-filter forwards to "some next handler," never to "the `CommunityModerator` class specifically." That means you can rewire the order or swap implementations freely.

Why the `nextHandler != null` check?

It's the guard for "I'm not the last handler." A non-terminal handler that can't resolve the report forwards it; but if it happens to be the tail (no successor set), the null check stops it from calling a method on `null`. The dedicated `SeniorModerator` makes the tail explicit, but the null guard keeps `AutoFilterModerator` / `CommunityModerator` safe regardless of where they sit.

Why a distinct terminal handler with an empty `setNextHandler` ?

To give the chain a **guaranteed end** and a **fallback**. `SeniorModerator` can't be linked further (empty setter, no field) and prints "Report could not be resolved by any moderator." when nothing matches. Without a terminal fallback, an unmatched report would just fall off the end of the chain unnoticed — here it's reported.

Why send every report only to the *first* handler?

That's the payoff of the pattern: the caller interacts with a single entry point and stays ignorant of the chain's length, order, and membership. The report propagates itself. In this demo `main` calls `autoFilterModerator.handleReport(...)` for all three severities, and each is actually resolved by a different, unnamed handler further down the chain.

Execution flow trace

```
main
├── build chain: autoFilter → community → senior
├── print "Chain Of Responsibility Design Pattern"
├── print "-- LOW severity report --"
│   ├── autoFilterModerator.handleReport( Report(LOW) )
│   │   ├── severity == LOW ? YES
│   │   └── prints "Auto-filter moderator resolved the report." ← consumed at auto-filter
├── print "-- MEDIUM severity report --"
│   ├── autoFilterModerator.handleReport( Report(MEDIUM) )
│   │   ├── severity == LOW ? no → escalate
│   │   └── prints "-----Escalating report to next moderator...CommunityModerator"
│   │       └── communityModerator.handleReport( Report(MEDIUM) )
│   │           ├── severity == MEDIUM ? YES
│   │           └── prints "Community moderator resolved the report." ← consumed at community
├── print "-- HIGH severity report --"
└── autoFilterModerator.handleReport( Report(HIGH) )
```

```

    severity == LOW ? no → escalate
    prints "-----Escalating report to next moderator...CommunityModerator"
    ↓
communityModerator.handleReport( Report(HIGH) )
    severity == MEDIUM ? no → escalate
    prints "-----Escalating report to next moderator...SeniorModerator"
    ↓
seniorModerator.handleReport( Report(HIGH) )
    severity == HIGH ? YES
    prints "Senior moderator resolved the report." ← consumed at senior

```

Note how the *same* call site in `main ( autoFilterModerator.handleReport(...) )` resolves at a different depth of the chain depending on the report's severity — that's the pattern working as intended.

Expected output

Captured from an actual run (`java -cp target/classes com.design.patterns.chainofresponsibility.ChainOfResponsibilityDesignPattern`):

```

Chain Of Responsibility Design Pattern
-- LOW severity report --
Auto-filter moderator resolved the report.
-- MEDIUM severity report --
-----Escalating report to next moderator:
com.design.patterns.chainofresponsibility.handler.impl.CommunityModerator
Community moderator resolved the report.
-- HIGH severity report --
-----Escalating report to next moderator:
com.design.patterns.chainofresponsibility.handler.impl.CommunityModerator
-----Escalating report to next moderator:
com.design.patterns.chainofresponsibility.handler.impl.SeniorModerator
Senior moderator resolved the report.

```

How to run

This module builds against JDK 11 (the reactor's other modules require it even though this one is plain Java). From inside this module directory:

```

JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
java -cp target/classes com.design.patterns.chainofresponsibility.ChainOfResponsibilityDesignPattern

```

Drop `-o` if Maven needs to resolve dependencies from a remote repository instead of the local cache.

Notes / possible improvements (not changed in the code)

- **Each handler stops at "resolved".** This is the *pure* Chain of Responsibility where exactly one handler consumes the report. A variant lets every handler act *and* forward (e.g. logging/validation pipelines) — that's the same pattern with the "stop" removed.
- **The chain is wired by hand** in `main` via `setNextHandler`. In a Spring service you'd typically inject the handlers (e.g. an ordered `List<IModerationHandler>`) and link them automatically, but hand-wiring keeps the mechanism visible here.
- Unlike some sibling modules, this one is a **plain main** with no `SpringApplication.run(...)` — the pattern needs no container.

Relationship to the other Behavioral patterns

- **Chain of Responsibility (this module)** passes a report along a line of handlers until one resolves it — the sender picks *nothing*, the chain decides.
- **Command** turns a request into an object you can queue/undo — it's about *what* to do, not *who* does it.
- **Strategy** picks *one* algorithm up front — no pass-along; the caller chooses the single handler directly.

Reach for Chain of Responsibility when **multiple objects might handle a request**, the handler set or order should be configurable, and the sender shouldn't be coupled to the specific receiver.